Ecole Nationale d'Ingénieurs de Sfax
Département de Génie Informatique
et de Mathématiques Appliquées

AU : 2025/2026
Niveau : GI 2

Technologies IOT: TP3

Prérequis :

Documentation

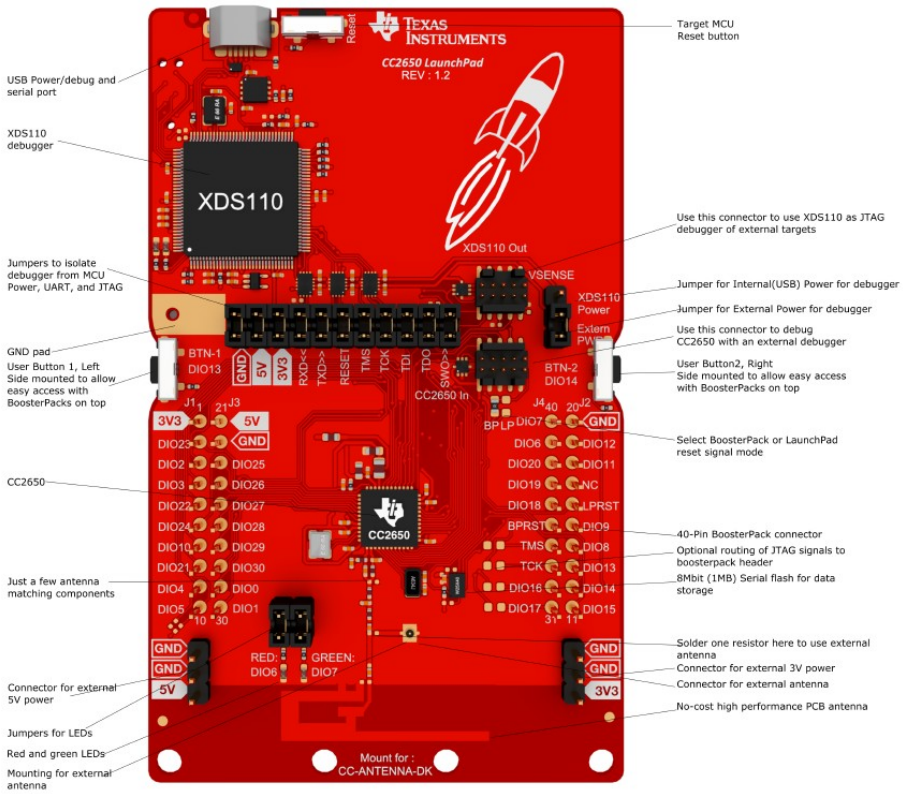
[1] Démarrer avec LaunchPad TI CC2650 et Contiki-ng

https://github.com/contiki-os/contiki/tree/master/platform/srf06-cc26xx

[2] SmartRF Flash Programmer 2 : un software Windows pour flasher l'exécutable sur les nœuds LaunchPad

https://www.ti.com/tool/FLASH-PROGRAMMER

Kit de développement IoT LaunchPad



Caractéristiques:

- System-on-Chip (SoC)
  - 32 bit ARM Cortex M3 @48MHz
  - 20 KB RAM
  - 128 KB code
- Radio

- Multi-standard: IEEE 802.15.4, Bluetooth LE, 6LowPAN
- On-board antenna

Commpilation

Définition du Target et Borad

Exemple avec le projet `hello-world`

```
make BOARD=launchpad/cc2650 TARGET=cc26x0-cc13x0 hello-world
```

=>Génération de fichiers exécutables (`*.cc26x0-cc13x0, *.elf, *.hex, *.bin, *.ihex`)

Remarque : si la compilation ne génère qu'un seul type d'exécutable (par exemple `*.cc26x0-cc13x0`), utilisez la commande suivante :

```
arm-none-eabi-objcopy -O binary hello-world.cc26x0-cc13x0 hello-world.bin
```

Cette commande necessite que la package `objcopy` soit installé

Pour vérifier : `objcopy --version`

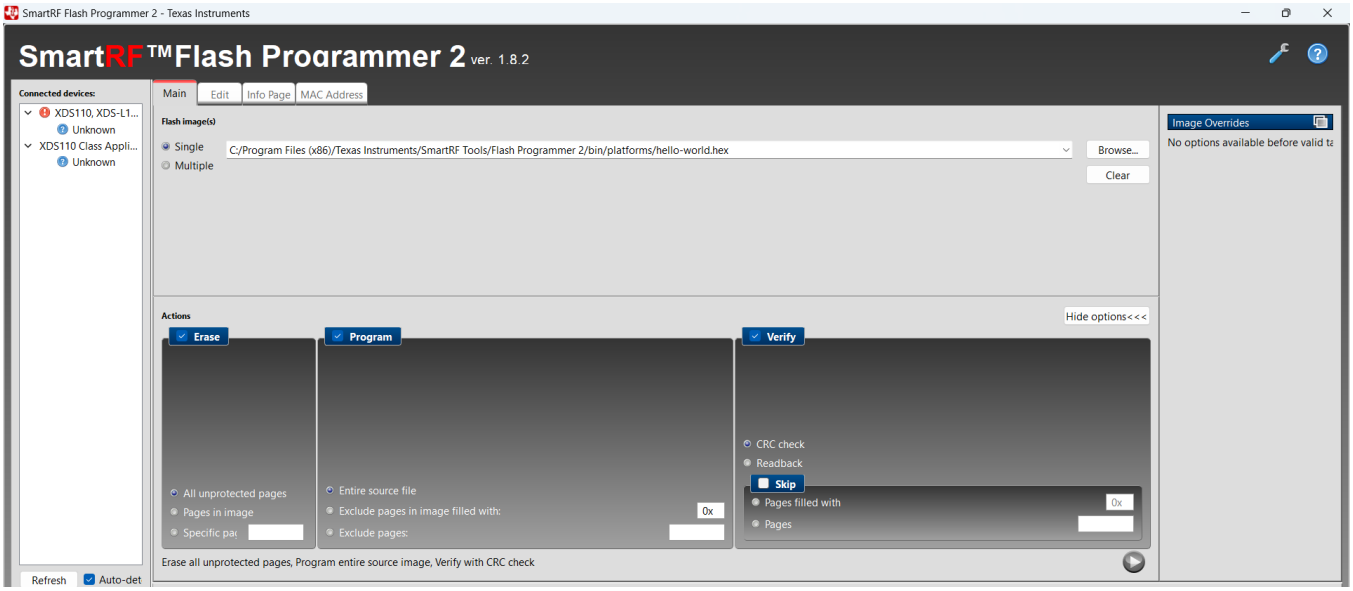
Pour installer : `sudo apt-get install binutils srecord`

Flasher le programme avec Flash Programmer 2

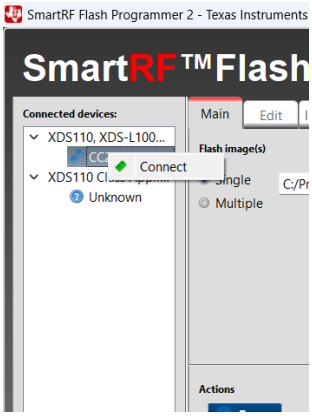
1) Brancher le nœud dans un port USB du PC

2) Copier le fichier exécutable (hex ou bin) à partir de la VM vers un emplacement sur Windows

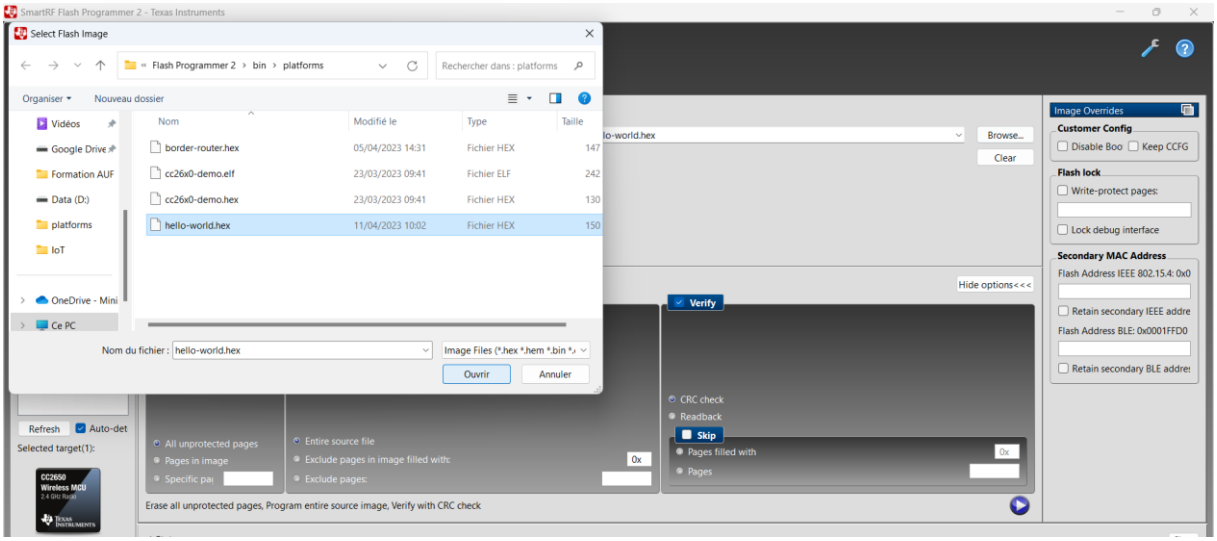
3) Lancer Flash Programmer 2



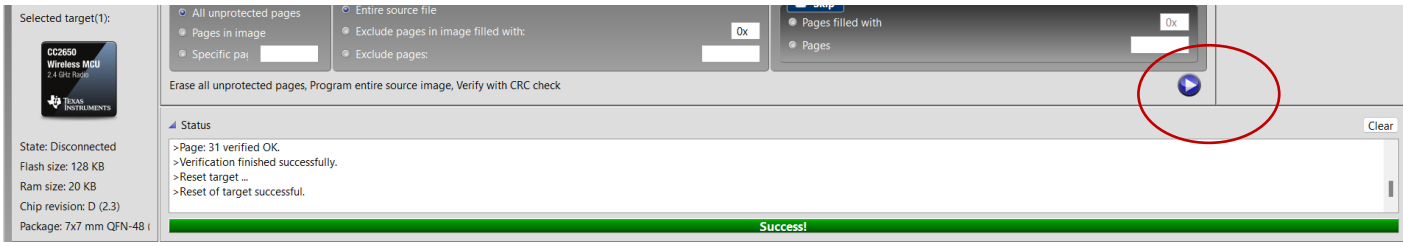
4) Se connecter avec le dispositif



5) Sélectionner le programme (le fichier exécutable importé de la VM)



6) Flasher



7) Tester

NB- Pour tester, il faut revenir à l'environnement Contiki-ng

>lsusb

```
nouha@ubuntu:~/contiki-ng/examples/hello-world$ lsusb
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 002 Device 005: ID 0451:bef3 Texas Instruments, Inc. CC1352R1 Launchpad
Bus 002 Device 004: ID 0e0f:0008 VMware, Inc. VMware Virtual USB Mouse
Bus 002 Device 003: ID 0e0f:0002 VMware, Inc. Virtual USB Hub
Bus 002 Device 002: ID 0e0f:0003 VMware, Inc. Virtual Mouse
Bus 002 Device 001: ID 1d6b:0001 Linux Foundation 1.1 root hub
```

>make login PORT=/dev/ttyACM0 TARGET=cc26x0-cc13x0

```
nouha@ubuntu:~/contiki-ng/examples/hello-world$ make login PORT=/dev/ttyACM0 TAR
GET=cc26x0-cc13x0
rlwrap ../../tools/serial-io/serialdump -b115200 /dev/ttyACM0
connecting to /dev/ttyACM0 [OK]
Hello, world
Hello, world
Hello, world
Hello, world
Hello, world
```

Objectifs du TP

L'objectif de ce TP est de mettre en pratique la communication IoT en utilisant la pile 6loWPAN. En particulier, nous allons utiliser le kit de développement IoT LaunchPad de Texas Instrument et le système d'exploitation Contiki-ng

Exercice 1 (adressage IPv6)

A) Nous souhaitons visualiser les adresses IPv6 des nœuds avec la commande `ip-addr`. Et pinger les nœuds du réseau IoT avec la commande `ping`.

Pour ce là il faut activer le NG Shell en ajoutant la ligne suivante dans le fichier makefile du projet hello-world (par exemple):

```
MODULES += os/services/shell
```

Compiler, flasher et tester sur les nœuds

```
[INFO: Main      ] - MAC: CSMA
[INFO: Main      ] - 802.15.4 PANID: 0xabcd
[INFO: Main      ] - 802.15.4 Default channel: 26
[INFO: Main      ] Node ID: 55295
[INFO: Main      ] Link-layer address: 0012.4b00.1204.d7ff
[INFO: Main      ] Tentative link-local IPv6 address: fe80::212:4b00:1204:d7ff
[INFO: CC26xx/CC13xx] TI CC2650 LaunchPad
[INFO: CC26xx/CC13xx]  RF: Channel 26, PANID 0xABCD
Hello, world
Hello, world
Hello, world

#0012.4b00.1204.d7ff> ip-addr
Node IPv6 addresses:
-- fe80::212:4b00:1204:d7ff
#0012.4b00.1204.d7ff> ip-nbr
Node IPv6 neighbors:
-- fe80::212:4b00:f8e:2885 <-> 0012.4b00.0f8e.2885, router 0, state Reachable
#0012.4b00.1204.d7ff>
```

```
[INFO: Main      ] Node ID: 10373
[INFO: Main      ] Link-layer address: 0012.4b00.0f8e.2885
[INFO: Main      ] Tentative link-local IPv6 address: fe80::212:4b00:f8e:2885
[INFO: CC26xx/CC13xx] TI CC2650 LaunchPad
[INFO: CC26xx/CC13xx]  RF: Channel 26, PANID 0xABCD
Hello, world
Hello, world
Hello, world

#0012.4b00.0f8e.2885> ip-addr
Node IPv6 addresses:
-- fe80::212:4b00:f8e:2885
#0012.4b00.0f8e.2885> ping fe80::212:4b00:1204:d7ff
Pinging fe80::212:4b00:1204:d7ff
Received ping reply from fe80::212:4b00:1204:d7ff, len 4, ttl 64, delay 15 ms
#0012.4b00.0f8e.2885> ip-nbr
Node IPv6 neighbors:
-- fe80::212:4b00:1204:d7ff <-> 0012.4b00.1204.d7ff, router 0, state Reachable
#0012.4b00.0f8e.2885>
```

B) Travail en groupes

Nous avons besoin de plusieurs nœuds ayant l'application hello-world avec le shell activé.

Définissez seul nœud comme racine du DAG avec la commande `rpl-set-root`.

```
> rpl-set-root 1
Setting as DAG root with prefix fd00::/64
```

Utilisez la commande `rpl-status` pour inspecter l'état RPL du nœud racine (root) :

```
> rpl-status
RPL status:
-- Instance: 0
-- DAG root
-- DAG: fd00::212:4b00:616:fcc, version 240
-- Prefix: fd00::/64
-- MOP: Non-storing
-- OF: MRHOF
-- Hop rank increment: 128
-- Default lifetime: 1800 seconds
-- State: Reachable
-- Preferred parent: (NULL IP addr)
-- Rank: 128
-- Lowest rank: 65535 (1024)
-- DTSN out: 240
-- DAO sequence: last sent 240, last acked 240
-- Trickle timer: current 13, min 12, max 20, redundancy 0
```

Tester la même commande sur le deuxième nœud

```
> rpl-status
RPL status:
-- Instance: 0
-- DAG node
-- DAG: fd00::212:4b00:616:fcc, version 240
-- Prefix: fd00::/64
-- MOP: Non-storing
-- OF: MRHOF
-- Hop rank increment: 128
-- Default lifetime: 1800 seconds
-- State: Reachable
-- Preferred parent: fe80::212:4b00:616:fcc
-- Rank: 489
-- Lowest rank: 489 (1024)
-- DTSN out: 240
-- DAO sequence: last sent 241, last acked 241
-- Trickle timer: current 12, min 12, max 20, redundancy 0
```

Expliquez….

Que fait la commande : `routes ?`

Exercice 2 (RPL)

Dans un nouveau dossier (par exemple TP3_ex2), reproduire le projet « rpl-udp » en apportant les modifications suivantes :

1) Dans le fichier `udp-client.c` Integrez votre nom ou pseudo dans le message (exemple : « Hello 1 from Zied »).

Le client Zied affiche :

[INFO: App] Sending hello 0 to fd00::c30c:0:0:1

Le serveur va donc afficher (on suppose avoir plusieurs 2 clients)

[INFO: App] Received 'hello 0 From Zied' fd00::c30c:0:0:4

[INFO: App] Received 'hello 1 From Hana' fd00::c30c:0:0:2

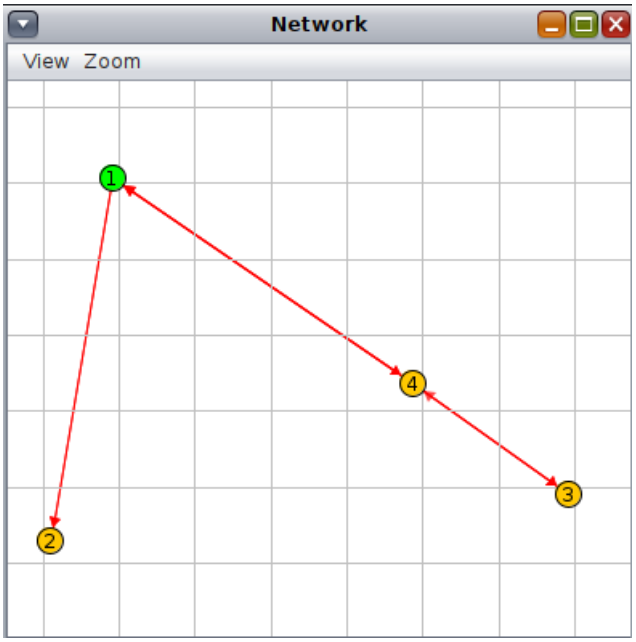
2) On souhaite que le serveur reçoit les messages des clients sans les répondre. Deux façons de faire soit commenter le code dans le fichier `udp-server` soit modifier la variable `WITH_SERVER_REPLY` => (`#define WITH_SERVER_REPLY 0`)
3) Activer les messages log de RPL

Rappel : pour se faire dans le fichier `project-conf.h` ajouter la ligne suivante :

```
#define LOG_CONF_LEVEL_RPL                    LOG_LEVEL_DBG
```

4) Activer le NG shell

Compiler et tester avec Cooja



Compiler, flasher et tester avec le kit IoT lanchpad

NB. Nous allons travailler en groupes de 2. Nous aurons donc 10 groupes donc 10 nœuds Launchpad.

Un groupe va jouer le rôle de serveur (DODAG root), les 9 groupes restants seront des clients.

Ajouter vos renseignements dans le tableau, exemple

pseudo	**Adresse IPv6**
DODAG root	fd00::212:4b00:1204:d7ff
Zied	fd00::212:4b00:f82:2885
…	

Analyser les messages log et tracer la topologie logique ainsi que la topologie physique, tout en indiquant les ranks.